

Template rapport technique – Assistant intelligent de recommandation d'événements culturels

1. Objectifs du projet

Contexte :

Puls-Events souhaite améliorer la manière dont les utilisateurs découvrent les événements culturels en proposant un assistant intelligent capable de comprendre des questions naturelles et de recommander des événements pertinents.

Problématique :

Les données d'événements sont nombreuses, variables et difficiles à interroger en langage naturel. Un système RAG permet d'exploiter ces informations efficacement grâce à la recherche sémantique et à un modèle de langage capable de synthétiser les réponses.

Objectif du POC :

Démontrer la faisabilité d'un assistant intelligent en combinant :

- une base vectorielle d'événements,
- un LLM pour générer des réponses précises,
- une API fonctionnelle interrogeable en direct.

Périmètre :

- Zone : France (ou la zone choisie dans les appels API OpenAgenda)
- Période (sur une durée d'environ un an de décembre 2024 à décembre 2025) : événements à partir des dates retournées par Données : titres, descriptions, dates, lieux, mots-clés, catégories

2. Architecture du système

Schéma global :

- **Données entrantes** : API OpenAgenda
- **Prétraitement** : nettoyage, normalisation, structuration

- **Embeddings** : vectorisation Mistral
- **Base vectorielle** : FAISS + métadonnées
- **LLM LangChain** : récupération contextuelle (retriever) + génération
- **API** : endpoints FastAPI /ask, /search, /health

Technologies utilisées :

Python 3.11, FastAPI, LangChain, FAISS, Poetry, Uvicorn, Mistral API, OpenAgenda API, GitHub Actions

3. Préparation et vectorisation des données

Source de données :

API OpenAgenda — événements récupérés via paramètres tels que : agenda UID, zone géographique, pagination.

Nettoyage :

- Suppression des champs vides
- Correction d'encodages
- Uniformisation des dates et textes

Chunking :

Découpage des descriptions pour améliorer la précision de la recherche — taille choisie : ~500 caractères avec un chevauchement de 50 pour garder le contexte.

Embedding :

- Modèle : Mistral Embeddings API
- Dimensions : 1024 (selon modèle)
- Vectorisation par batchs
- Stockage dans FAISS + fichier JSON des métadonnées

4. Choix du modèle NLP

Modèle sélectionné :

Mistral Large

Pourquoi ce modèle ?

- Coût faible
- Intégration directe avec LangChain
- Stable, adapté aux tâches de QA contextuelle

Prompting :

- Instruction simple : "Tu es un assistant culturel. Donne une réponse claire, concise et utile "

Limites du modèle :

- Dépendance à la qualité des données
- Peut halluciner si aucun contexte n'est trouvé
- Peut manquer de précision en terme de périodicité (date) faute du manque de données

5. Construction de la base vectorielle

FAISS utilisé :

Index FlatL2 (simple et efficace pour un POC)

Stratégie de persistance :

- Index sauvegardé dans `faiss_indexes` (`index_description.index`, `index_description_longues.index`)
- Métadonnées dans `faiss_indexes` (`index_description.metadata.pkl`, `index_description_longues.metadata.pkl`)

Métadonnées associées :

- Titre de l'événement
- Description (embédé et indexé)
- Description longue (embédé et indexé)
- Date début/ data fin
- Ville
- Latitude/longitude
- Mot clé
- ID d'événement

6. API et endpoints exposés

Framework utilisé :

FastAPI

Endpoints clés :

- GET / → message d'accueil
- GET /search → recherche vectorielle simple
- POST /ask → pipeline complet RAG (retrieval + génération)
- POST /rebuild → reconstruction de l'index depuis la collecte des données
- GET /health → statut du service

Format des requêtes/réponses :

JSON standard :

- input:{"question": "...", "k": 3} Où k correspond au nombre de meilleur résultat après filtrage des résultats obtenue sur les indexes des descriptions
- output:{"question": "...", "generated_answer": "...", "contexts": [...]}

Exemple d'appel (curl) :

```
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" -d '{"question": "Que faire à Paris ?", "k": 3}'
```

Tests effectués :

Tests unitaires via pytest (api, endpoints)

Gestion des erreurs :

- Exceptions capturées
- Messages explicites si aucun résultat n'est trouvé
- Validation des paramètres FastAPI

7. Évaluation du système

Jeu de test annoté :

- ~20 à 30 questions typiques d'utilisateurs
- Annotation manuelle : pertinence des réponses

Métriques :

- Similarité sémantique (cosinus)
- Taux de couverture (réponses avec contexte)
- Analyse qualitative (pertinence perçue)

Résultats :

- Bonnes réponses sur les événements bien décrits
- Quelques limites sur les événements très courts ou mal renseignés, et parfois sur les dates.

8. Recommandations et perspectives

Ce qui fonctionne bien :

- Recherche sémantique fluide
- Réponse contextuelle efficace
- Réponses pertinentes sur les événements riches en texte
- API stable et facile à intégrer

Limites du POC :

- Données parfois incomplètes
- Absence de filtrage de temps avancé
- RAG sensible à la qualité des embeddings

Améliorations possibles :

- Ajouter des filtres (date, type d'événement, prix)
- Embeddings plus performants (réduction de la longueur des chunks pour plus d'efficacité)
- Refinement du prompt

9. Organisation du dépôt GitHub

Arborescence :

- `scripts/` : scripts principaux (RAG, embeddings, collecte de données)

- `data/` : stockage local des données
- `tests/` : tests automatisés
- `faiss_index/` : index vectoriel
- `pyproject.toml` : dépendances Poetry
- `.github/workflows/` : CI/CD

Description rapide :

- `rag_fast_api.py` → API principale
- `generate_embeddings.py` → vectorisation
- `data_collected.py` → récupération OpenAgenda
- `index_faiss_metadata.py` → construction FAISS

10. Annexes

- Exemples du jeu d'évaluation
- Exemples de réponses brutes JSON
- Schéma UML de l'architecture du système